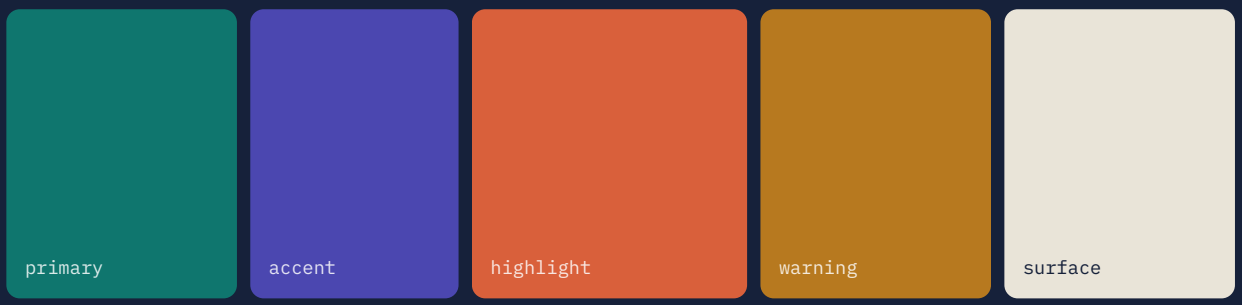
A PLAIN-LANGUAGE GUIDE FOR UX ARCHITECTS & DEV TEAMS

One Design System, Every Stack

How to turn a single design system into one package that works with Angular, React, Vue, Tailwind, Bootstrap and Material UI, explained step by step, without assuming any technical background.



Contents · The basic words · The problem · The solution · Building it in 10 steps · Plugging in each project · Effort, timeline & rules

PART 1

First, the basic words

Before the solution makes sense, it helps to know what each term means. Here they are in everyday language, each with a comparison from ordinary life.

Application	Any program you use on a computer or phone, like a banking website or a shopping app.
Development	The work of building an application. Developers write instructions (code) that tell the computer what to show and what to do.
Design	Deciding how an application looks and feels: colors, fonts, button sizes, spacing.
Design system	A company's rulebook plus a box of ready-made parts for building screens, so every app looks like part of the same family. Like a coffee chain: every branch gets the same colors, cups and menu boards from head office.
Tokens	Named design choices. Instead of "this exact blue," you say "Brand Blue." Change it once, and it changes everywhere. Like the paint names on a paint card.
Components	Ready-made screen parts built from tokens: a button, a search box, a pop-up window, a menu. Like pre-built furniture from a warehouse.
Building toolkits	Angular, React and Vue. They decide how an application is constructed. Parts from one don't fit another. Like building with bricks versus steel frames.
Decorating kits	Tailwind, Bootstrap, Material UI and Vuetify. They decide how things look, each with its own colors and styles. Like different interior-decorating brands.
npm package	A box of code that any team can download with one command. Like a parcel with labeled compartments inside.

PART 2

The problem

You have one design system, but every project is built with a different combination of toolkits.

PROJECT A

Angular + Tailwind

Bricks | Tailwind kit

PROJECT B

React + Bootstrap

Steel | Bootstrap kit

PROJECT C

React + Material UI

Steel | Material kit

Every decorating kit arrives with its own idea of what "blue" is and what a "button" looks like. So for each project, someone has to manually repaint and rebuild your design inside a different kit. That's the back-and-forth you keep running into.

> It's like a coffee chain where each branch was handed a different paint brand and told "make it look like head office," and someone mixes the colors by hand, every single time.

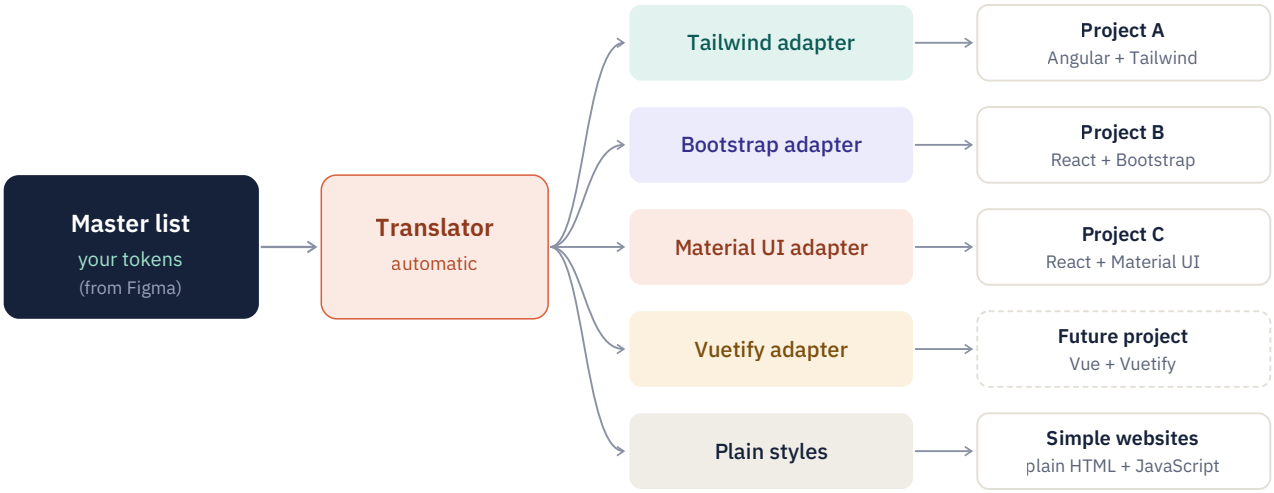
What this costs you

Colors drift slightly between projects, the same work is repeated three times, and every brand change means hunting through three codebases.

PART 3

The solution, in one picture

Think of a phone charger and travel adapters. The phone never changes; you simply carry an adapter for each country's socket.



One master list feeds an automatic translator, which writes an adapter for every decorating kit.

The five ideas behind it

#	IDEA	WHAT IT MEANS IN PRACTICE
1	**One master list**	All your named choices (Brand Blue, Medium Space, Heading Font) live in one file. Nobody edits colors anywhere else.
2	**Automatic translation**	A tool reads the master list and writes a version for Tailwind, Bootstrap, Material UI, Vuetify, and plain styling. Change once, press a button, everything updates.
3	**Repaint, don't replace**	Teams keep the kit they know. Bootstrap's buttons stay Bootstrap's buttons, but painted in your colors and shapes.
4	**Universal special parts**	Components unique to you are built once as web components, which every browser understands, then wrapped for Angular, React and Vue.
5	**One box, many compartments**	Everything ships as a single npm package. Each project opens only the compartments it needs.

> **One package, not three**
>
> You don't publish a separate package per framework. It's one travel kit with several adapters inside. Projects only pull out what they use, so an Angular app never downloads React, and unused compartments never make an app heavier.

Is Material UI available for Vue?

No. Material UI works only with React. For Vue, the equivalent Material-style kit is **Vuetify**. That's no problem for this approach: you simply add a Vuetify adapter fed by the same master list. Whenever a new kit appears, you add one adapter instead of redoing your design.

PART 4

Building it, step by step

From a design system on your laptop to one package every team can install. Each step shows who is responsible.

1 Take stock of what you have

You

Gather every color, font, font size, spacing value, corner roundness, shadow and icon. Then list your components and mark each one either **"the kits already have this"** (buttons, inputs, tables) or **"unique to us"** (say, a special pricing card). Only the unique ones need building from scratch later.

Also write down every combination to support today, plus planned ones.

2 Turn your choices into named tokens

You + designer

In Figma, use Figma Variables or the Tokens Studio plugin to name every choice, organized in three levels:

- **Raw values**, the full paint card: "Blue 600 = this exact blue."
- **Meanings**, what each paint is for: "Primary action color = Blue 600."
- **Part-specific** (optional): "Button background = Primary action color."

Projects always use **meanings**, never raw paint. If your brand moves from blue to green, you change one link. Tokens Studio exports this as a simple text file:

```
{
  "color": {
    "blue-600": { "value": "#185FA5" },
    "primary":  { "value": "{color.blue-600}" }
  },
  "space": { "medium": { "value": "16px" } }
}
```

3 Create the box

Developer

A developer creates one project folder, stored on GitHub so changes are tracked. Free tools: Node.js, pnpm and Git.

```
your-design-system/
├── tokens/        ← your master list from Figma
├── adapters/      ← translator settings
├── components/    ← your unique components
├── docs/          ← preview gallery
└── dist/          ← generated compartments (automatic)
```

4 Set up the automatic translator

Developer

Install **Style Dictionary** and configure it to write one file per kit. After this, one command (`pnpm build`) regenerates everything. This is the moment hand-copying disappears.

COMPARTMENT	BECOMES	USED BY
Plain styles	CSS variables like `--ds-color-primary`	Every project
Tailwind	A Tailwind theme file	Project A
Bootstrap	A Bootstrap settings file	Project B
Material UI	A Material UI theme	Project C
Vuetify	A Vuetify theme	Future Vue projects

Developer note: Bootstrap and Material UI calculate shades internally (like a darker hover color), so their files should contain actual color values. Style Dictionary handles this automatically.

5 Build only your unique components

Developer + you review

Take only the "unique to us" list from Step 1 and build each component once with **Stencil**. It creates web components and automatically produces matching versions for React, Angular and Vue. Components the kits already have aren't rebuilt; the adapters simply repaint them.

6 Label the compartments and package it

Developer

Set up the package's label list so each compartment has its own name, all inside one package:

```
@yourname/ds/tailwind     @yourname/ds/react
@yourname/ds/bootstrap    @yourname/ds/angular
@yourname/ds/mui          @yourname/ds/vue
@yourname/ds/vuetify      @yourname/ds/styles.css
```

Two safety rules: prefix everything with `ds` so nothing clashes with Tailwind or Bootstrap names, and make every toolkit optional so projects download only what they use.

7 Build a preview gallery

Developer sets up, you own content

Set up **Storybook**, a website where every token and component can be viewed and tested, with a page per combination showing how to plug in. It becomes the single place teams go for answers.

---

8 Publish the package

Developer

Publish privately via an npm organization, GitHub Packages, or a company registry. Each release gets a version number (1.0.0, 1.1.0) and **Changesets** writes the "what changed" notes automatically.

---

9 Plug it into one project first

Project developers

Pilot with one project, perhaps Project B, since repainting Bootstrap is usually the quickest win. Fix what the pilot reveals, release a new version, then roll out to the rest. See the next page for how each combination plugs in.

---

10 Agree on how changes happen

You own this

Write down a simple routine and one firm rule (both on the final page). This is what keeps the system trustworthy over time.

PART 5

Plugging in each project

Every team installs the same package with one command, then opens the compartments matching its combination.

```
# the same for every project
npm install @yourname/ds
```

PROJECT	OPENS THESE COMPARTMENTS	WHAT THE TEAM DOES
A · Angular + Tailwind	`tailwind` + `angular`	Adds the Tailwind compartment to the main style file; imports unique components from the Angular compartment.
B · React + Bootstrap	`bootstrap` + `react`	Loads the Bootstrap compartment before Bootstrap itself, so Bootstrap picks up your colors.
C · React + Material UI	`mui` + `react`	Wraps the app in Material UI's theme provider using your theme.
Future · Vue + Vuetify	`vuetify` + `vue`	Passes your theme in when setting up Vuetify.
Plain websites	`styles.css` + components	Links one stylesheet; uses components as simple tags like `<ds-modal>`.

> **The result**
>
> Three projects, three different toolkit combinations, all looking like one family, because all of them trace back to the same master list.

Avoiding clashes

Prefix everything

Use `ds` on every class, variable and component name so nothing collides with `.btn` or Tailwind utilities.

Use style layers

Wrap your styles in a CSS cascade layer, so Tailwind's or Bootstrap's base resets can't unpredictably override them.

Don't fight the kit

Re-theme the existing kit with your tokens rather than adding a parallel, competing set of components.

PART 6

Effort, timeline & rules

A little more work upfront, and much less every time after that.

Where teams save effort

- No more hand-copying colors, fonts and spacing into each project.
- A brand change is made once and reaches every project with an update.
- New projects start faster, with the look ready from day one.
- Fewer "that's not the right shade" debates between design and development.

Where effort still exists

- Initial setup is a real investment, paying off from the 2nd or 3rd project.
- Adapters need updating when kits release major new versions.
- Kits have slightly different parts and behaviors; tokens unify the look, not every behavior.
- Someone must own it, likely you as UX architect.

Rough timeline (one developer working with you)

1–2 weeks
Steps 1–4 · tokens & adapters. Every project already gets your look.

2–4 weeks
Step 5 · unique components, depending on how many you have.

~1 week
Steps 6–9 · package, gallery, publish & pilot.

Value starts after the first stage, so teams don't need to wait for everything.

The change routine

Design Change starts in Figma → **Approve** You update the master list → **Build** Developer runs the translator → **Publish** New version released → **Update** Projects install it

The one golden rule

Nobody types colors or spacing directly into project code. Everything comes from the package.

What you hand your development team

- [ ] The master token list: your design choices, named and organized in three levels.
- [ ] The instruction to set up the translator for Tailwind, Bootstrap, Material UI, Vuetify and plain styles.
- [ ] The list of unique components to build once as web components.
- [ ] The rule that every project installs the single package instead of copying values by hand.